Politechnika Wrocławska

Akceleracja obliczeń w przetwarzaniu danych

Porównanie wydajności mnożenia macierzy na CPU i GPU

Raport

Michał Musielak 281092

Klaudyna Kijek 280891

Mikołaj Brzuśnian 280895

Yuliia Nudha 282876

Prowadzący: Michał Zając

Spis treści

1 Cel projektu **2**

2 Przyjęte założenia i środowisko testowe **2**

3 Wstęp teoretyczny **2**

4 Implementacja **3**
4.1 Trzy wersje algorytmu 3
4.2 Sprawdzanie poprawności 3

5 Metodologia przeprowadzania badań **4**

6 Wyniki i wnioski **4**
6.1 Czas wykonania i przyspieszenie 4
6.2 Skalowalność CPU i zmienność pomiarów 6
6.3 Profilowanie GPU 7

7 Podsumowanie **7**

1 Cel projektu

Celem projektu było porównanie czasu klasycznego mnożenia macierzy w trzech wersjach: sekwencyjnej na CPU, równoległej na CPU i równoległej na GPU. Zbadano wpływ rozmiaru macierzy i liczby wątków na wydajność oraz wykorzystano profiler do analizy pracy GPU.

2 Przyjęte założenia i środowisko testowe

Program napisano w C++17 z użyciem `std::thread` i CUDA 13.4. Kompilację wykonano z optymalizacją `-O3`. Macierze kwadratowe zapisano wierszami, używając typu `float`. Wszystkie wersje realizują ten sam algorytm i otrzymują te same dane. Nie korzystano z gotowych bibliotek do mnożenia macierzy.

Tabela 1: Środowisko testowe

Element	Specyfikacja
CPU	AMD Ryzen 7 3700X, 8 rdzeni, 16 wątków
GPU	NVIDIA GeForce RTX 3060 Ti, 8 GiB
RAM	31,26 GiB
System	CachyOS Linux

Na czas badań ograniczono liczbę działających aplikacji. Zegar GPU ustawiono na 1500 MHz, a pamięci na 6801 MHz opcjami `-lgc` i `-lmc` narzędzia `nvidia-smi` [1]. Monitoring potwierdził te wartości. Dla CPU wybrano tryb `performance`, zadano 3,60 GHz i wyłączono boost. Odczyty CPU wynosiły 3,36–3,61 GHz; żądana częstotliwość nie gwarantuje idealnie stałego taktowania [2]. Program mógł korzystać z ośmiu rdzeni fizycznych. Po badaniach przywrócono poprzednie ustawienia komputera.

3 Wstęp teoretyczny

Iloczyn macierzy A i B o rozmiarze $N \times N$ wyznacza wzór

$$c_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{kj}. \tag{1}$$

Każdy element wyniku jest sumą iloczynów elementów wiersza A i kolumny B. Przykładowo:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$

Mnożenie macierzy wykorzystuje się m.in. w grafice i obliczeniach naukowych. Klasyczny algorytm ma złożoność $O(N^3)$.

Elementy wyniku są niezależne, dlatego można obliczać je równolegle. CPU dzieli pracę między kilka rdzeni, a GPU wykonuje wiele podobnych zadań jednocześnie. Korzyść z równoległości ograniczają koszty obsługi wątków, dostępu do pamięci oraz przesyłania danych do GPU. Przy małych macierzach koszty te mogą przewyższać oszczędność czasu samych obliczeń.

4 Implementacja

4.1 Trzy wersje algorytmu

Wersja sekwencyjna wykonuje trzy pętle: po wierszach, kolumnach i składnikach sumy. Wersja równoległa CPU dzieli wiersze wyniku między wątki `std::thread`. Wątki zapisują rozłączne części macierzy, więc nie wymagają blokad. Przed odebraniem wyniku program czeka na zakończenie wszystkich wątków.

W wersji CUDA jeden wątek oblicza jeden element wyniku. Wątki uruchamiane są w blokach po 256. Poniżej przedstawiono kod jądra GPU:

```
__global__ void gpu_kernel(const float* a,
    const float* b, float* c, int n) {
    int index = blockIdx.x * blockDim.x
              + threadIdx.x;
    if (index >= n*n) return;
    int i = index / n;
    int j = index % n;
    float sum = 0;
    for (int k = 0; k < n; ++k)
        sum += a[i*n+k] * b[k*n+j];
    c[index] = sum;
}
```

Przed obliczeniami dane są kopiowane do GPU, a po nich wynik wraca do pamięci CPU. Ważne było poprawne obsłużenie niepełnego bloku GPU, nierównego podziału wierszy CPU oraz zakończenia pracy GPU przed zatrzymaniem pomiaru czasu.

4.2 Sprawdzanie poprawności

Sprawdzono znany iloczyn 2×2, macierze zerowe, jednostkowe, wypełnione jedynkami oraz dane losowe, ułamkowe i ujemne. Testy obejmowały również rozmiary 17 i 33, które nie dzielą się równo między wszystkie badane wątki. Wszystkie dziewięć przypadków przeszło testy w każdej wersji.

Dopuszczalną różnicę zmiennoprzecinkową określono warunkiem

$$|c - c_{\mathrm{ref}}| \le 10^{-3} + 10^{-4}|c_{\mathrm{ref}}|. \tag{2}$$

Dodatkowe porównanie z niezależnym wynikiem typu `double` objęło 108 sprawdzeń dla rozmiarów od 1 do 513. Wszystkie były poprawne; największy błąd bezwzględny wyniósł $3{,}52 \cdot 10^{-5}$.

5 Metodologia przeprowadzania badań

Macierze wypełniano liczbami pseudolosowymi z przedziału $[-1, 1]$, ze stałym ziarnem 42. Zbadano rozmiary $N = 64$, 128, 256 i 512 oraz CPU równoległy z 1, 2, 4 i 8 wątkami. Wykonano trzy pełne serie. W każdej konfiguracja miała co najmniej sekundę rozgrzewki i 20 prób. Wyniki przedstawiono jako średnie z 60 prób; nie odrzucano wolniejszych pomiarów.

Czas mierzono zegarem monotonicznym biblioteki standardowej C++. Dla CPU równoległego uwzględniono tworzenie i zakończenie wątków. Dla GPU pomiar obejmował rezerwację pamięci, transfery, obliczenia, synchronizację i zwolnienie pamięci. Generowanie danych i kontrola wyników odbywały się poza pomiarem. Wszystkie 1440 prób głównej serii dało poprawny wynik.

Przyspieszenie względem wersji sekwencyjnej obliczono jako

$$S = \frac{\bar{t}_{\text{seq}}}{\bar{t}_{\text{wariant}}}. \tag{3}$$

Na wykresach punkty oznaczają średnie, a przerywane linie dopasowany trend potęgowy. Słupki na wykresach czasu pokazują jedno odchylenie standardowe. Punkty nie są połączone. Wyniki zaokrąglono do dwóch miejsc po przecinku po wykonaniu obliczeń.

GPU profilowano osobno narzędziem NVIDIA Nsight Systems dla $N = 512$. Analiza dotyczyła 20 prób po rozgrzewce. Czasów z tego uruchomienia nie używano do obliczania przyspieszeń ani głównych wykresów.

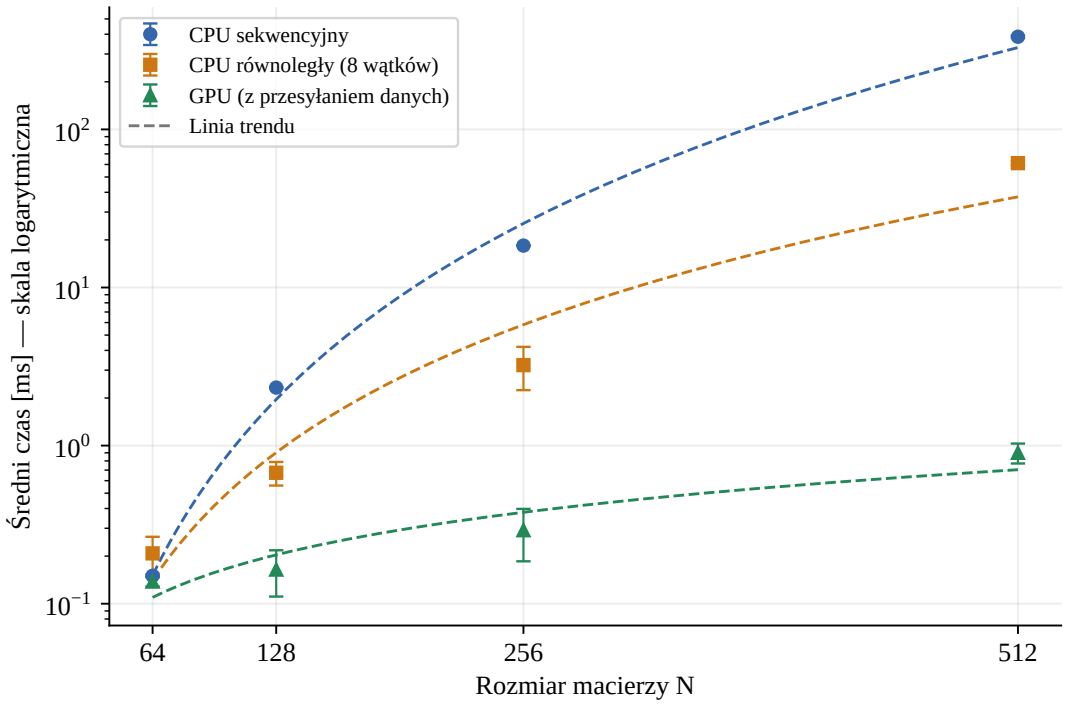
6 Wyniki i wnioski

6.1 Czas wykonania i przyspieszenie

Tabela 2 i rysunek 1 porównują trzy wersje algorytmu. W tym zestawieniu CPU równoległy używa 8 wątków.

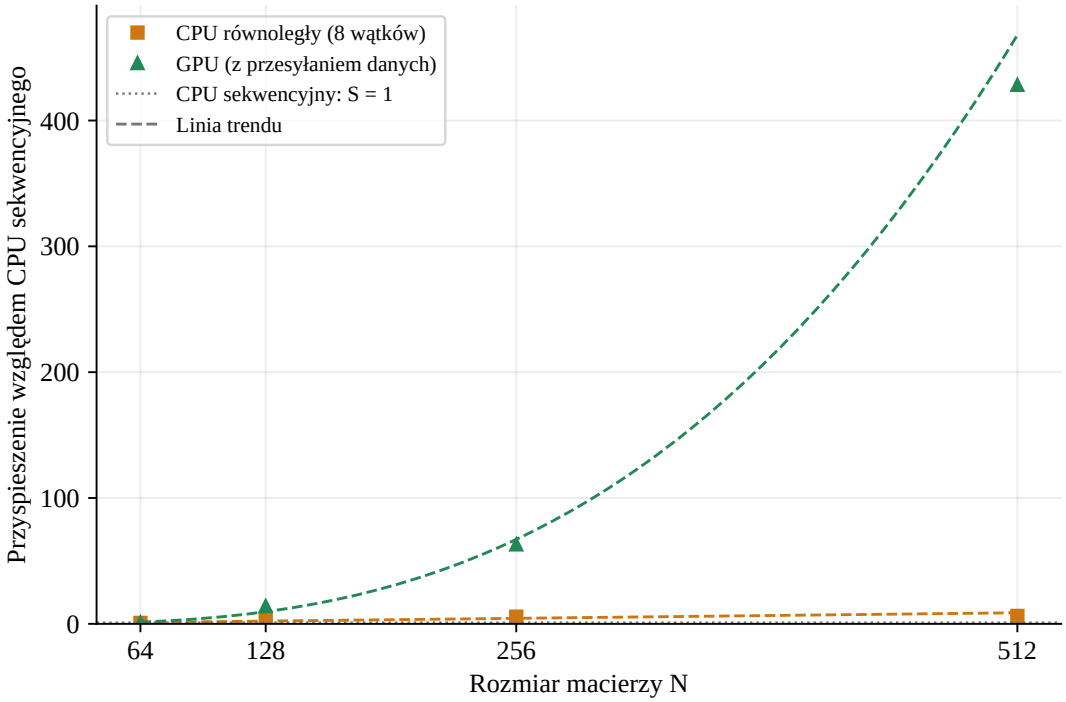
Tabela 2: Średni czas mnożenia [ms]

N	CPU sekw.	CPU, 8 wątków	GPU
64	0,15	0,21	0,14
128	2,33	0,67	0,16
256	18,41	3,23	0,29
512	385,62	61,18	0,90



Rysunek 1: Czas wykonania w zależności od rozmiaru macierzy.

Wraz ze wzrostem macierzy zwiększał się czas wszystkich wersji. Dla $N = 512$ GPU potrzebowało 0,90 ms, CPU z 8 wątkami 61,18 ms, a CPU sekwencyjny 385,62 ms. Odpowiada to przyspieszeniu 428,32 razy dla GPU i 6,30 razy dla CPU równoległego, co przedstawia rysunek 2. Wyniki odnoszą się do prostej implementacji i podanych warunków badań.

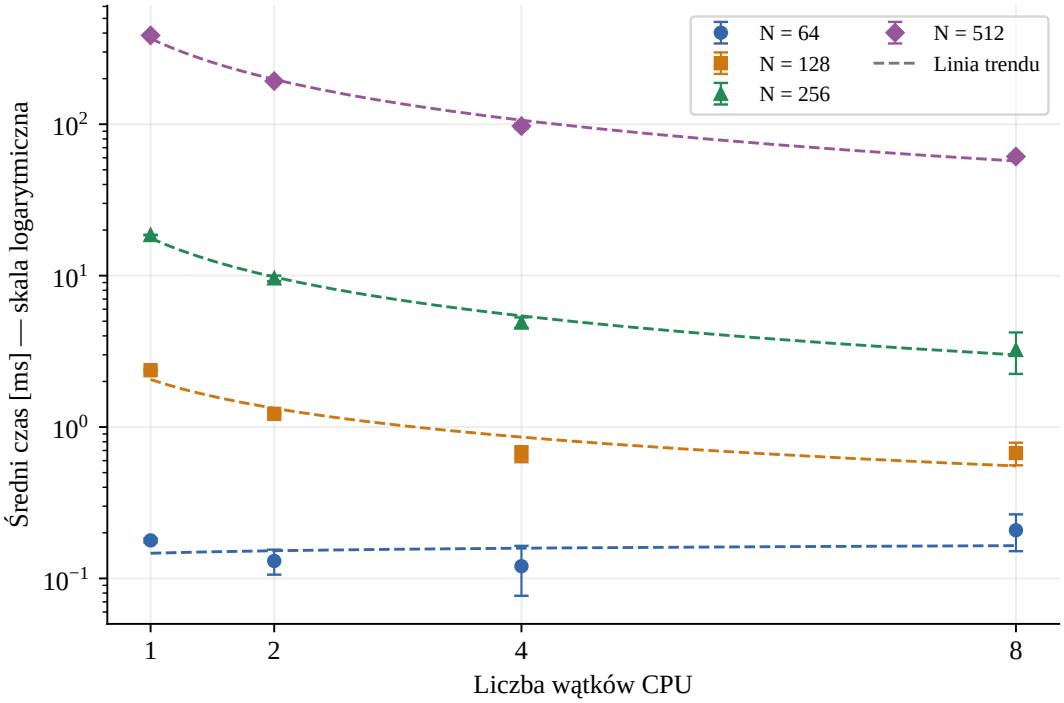


Rysunek 2: Przyspieszenie względem CPU sekwencyjnego.

Dla $N = 64$ CPU z 8 wątkami było wolniejsze od wersji sekwencyjnej, a przewaga GPU była niewielka. Pokazuje to, że równoległość daje większe korzyści, gdy obliczeń jest wystarczająco dużo.

6.2 Skalowalność CPU i zmienność pomiarów

Rysunek 3 pokazuje wpływ liczby wątków. Dla $N = 512$ zwiększenie ich liczby z 1 do 8 skróciło czas z 386,06 do 61,18 ms. Dla $N = 64$ najlepsze były 4 wątki z czasem 0,12 ms, krótszym także od GPU. Przy małych zadaniach koszt obsługi kolejnych wątków ogranicza skalowanie.



Rysunek 3: Czas wykonania CPU równoległego w zależności od liczby wątków.

Dodatkowe próby dla $N = 511$, 512 i 513 potwierdziły spowolnienie CPU dla $N = 512$. Może ono wynikać z kolumnowego odczytu macierzy B i działania pamięci podręcznej; nie potwierdzono jednak tej przyczyny pomiarem chybień. Złożoność $O(N^3)$ opisuje liczbę operacji, a nie stały czas każdej z nich.

Ustawienia zegarów nie usunęły całej zmienności. Dla GPU przy $N = 512$ odchylenie standardowe wyniosło 0,13 ms, czyli 14,40% średniej. Pełny czas obejmuje też sterownik, transfery i obsługę pamięci. Nie ustalono osobnego wpływu tych czynności na rozrzut.

6.3 Profilowanie GPU

Tabela 3 zawiera wybrane wyniki Nsight Systems. Czasy opisują pracę urządzenia, a nie całe wywołanie programu.

Tabela 3: Profil GPU dla $N = 512$: średni czas z 20 prób [ms]

Czynność	Czas
Obliczenia na GPU	0,45
Przesłanie obu macierzy do GPU	0,09
Odebranie wyniku z GPU	0,05

Jądro zajmowało 76,33% łącznego czasu obliczeń i transferów na urządzeniu. Profil wykazał też koszt powtarzanej rezerwacji i zwalniania pamięci. Ponowne użycie buforów mogłoby go ograniczyć, ale takiej optymalizacji nie wprowadzono.

7 Podsumowanie

Przygotowano i sprawdzono trzy wersje mnożenia macierzy. Dla większych badanych rozmiarów GPU było najszybsze, a wielowątkowość skróciła czas CPU. Dla małych macierzy koszt uruchamiania pracy równoległej ograniczał korzyści. Pomiary, analiza skalowalności i profilowanie pozwoliły osiągnąć cel projektu.

Literatura

[1] NVIDIA, *Dokumentacja nvidia-smi*, `https://docs.nvidia.com/deploy/nvidia-smi/index.html`, dostęp: 01.11.2026.

[2] The Linux Kernel, *CPU Performance Scaling*, `https://docs.kernel.org/admin-guide/pm/cpufreq.html`, dostęp: 01.11.2026.